گزارش تمرین دوم

پلتفرم بانکداری توزیع‌شده

۱۵ آبان ۱۴۰۵ • درس سیستم‌های توزیع‌شده • شاردینگ، رپلیکیشن و تراکنش دوفازی

چکیده. در این پروژه یک سیستم بانکداری توزیع‌شده برای ۲۰۰۰ حساب پیاده‌سازی شده است. حساب‌ها بین دو شارد تقسیم می‌شوند و هر شارد دو رپلیکای برنامه دارد که از یک پایگاه‌داده مشترک استفاده می‌کنند. کلاینت فقط با گیت‌وی در ارتباط است و گیت‌وی بر اساس نقشه شارد، درخواست را به شارد مالک و سپس به یک رپلیکای سالم می‌فرستد. برای انتقال بین شاردها از پروتکل دوفازی (2PC) با ژورنال پایدار استفاده شده تا تصمیم تراکنش پس از ری‌استارت هم قابل بازیابی باشد. نقشه شارد دارای اپوک است و اپوک در خود شارد هم بررسی می‌شود تا درخواست‌های قدیمی پس از تغییر مالکیت اثر نگذارند. در بخش امتیازی اول، شارد C اضافه می‌شود و بازه A1501-A2000 در حین سرویس‌دهی از B به C منتقل می‌شود. در بخش دوم، یک کنترلر ساده خرابی رپلیکاها را تشخیص می‌دهد و آن‌ها را روی همان پایگاه‌داده شارد جایگزین می‌کند. مجموعه تست پروژه ۲۷ بررسی دارد و همه آن‌ها در اجرای نهایی قبول شدند.

۱. نمای کلی سیستم

این سیستم برای حساب‌های A0001–A2000 ساخته شده است. داده‌ها به دو شارد تقسیم می‌شوند: شارد A حساب‌های A0001–A1000 و شارد B حساب‌های A1001–A2000 را در اختیار دارد. هر شارد دو رپلیکای برنامه دارد (A1، A2 و B1، B2) که هر دو به یک پایگاه‌داده مشترک وصل می‌شوند. در این گزارش، منظور از «شارد» یک بخش از داده است و منظور از «رپلیکا» یک فرایند سرویس‌دهنده روی همان داده. کلاینت بانکی فقط آدرس گیت‌وی را می‌داند و انتخاب شارد و رپلیکا کار گیت‌وی است. اگر دو حساب یک انتقال در دو شارد مختلف باشند، گیت‌وی درخواست را به هماهنگ‌کننده تراکنش می‌دهد تا 2PC اجرا شود. ناظر نقشه شارد هم نقشه پایدار و مهاجرت را اداره می‌کند و کنترلر ترمیم مسئول جایگزینی رپلیکاهای خراب است.

مؤلفه	پیاده‌سازی	مسئولیت
کلاینت بانکی	platform/client.py	موجودی، واریز، برداشت و انتقال؛ هر نوشتن را با request_id ثابت دوباره تلاش می‌کند؛ تراکنش نامعلوم را با شناسه کوئری می‌کند. فقط یک URL می‌شناسد: گیت‌وی.
گیت‌وی آگاه به شارد	platform/gateway.py	شارد مالک را از بازه حساب انتخاب می‌کند، سپس رپلیکا را با کمترین اتصال؛ سلامت رپلیکاها را بررسی می‌کند؛ انتقال بین‌شاردی را به هماهنگ‌کننده می‌فرستد؛ هرگز حساب را به شارد غیرمالک نمی‌فرستد.
رپلیکاهای شارد	platform/shard_server.py، platform/storage/shard_store.py	خواندن و نوشتن، انتقال درون‌شاردی اتمیک، مشارکت در 2PC و نقاط مهاجرت را سرو می‌کنند. هر رپلیکا همان پایگاه‌داده شارد را باز می‌کند.
هماهنگ‌کننده تراکنش	platform/coordinator.py، platform/storage/registry.py	تخصیص txn_id؛ اجرای prepare/commit یا abort؛ ثبت هر گذار قبل از اقدام؛ بازیابی تراکنش‌های حل‌نشده پس از ری‌استارت.
ناظر نقشه شارد	platform/shardmap_service.py	نقشه پایدار، تاریخ اپوک و رجیستری اکتورها؛ انتشار و پوش نقشه؛ ثبت رپلیکا؛ شروع و ادامه مهاجرت؛ رابط HTTP عملیاتی.
کنترلر خودترمیم	platform/selfhealing.py	بررسی رپلیکاهای ثبت‌شده، تشخیص خرابی، ادعای پایدار یک جایگزین، ساخت جایگزین روی همان والیوم و ثبت آن در نقشه.
هارنس محلی	platform/local_stack.py	همان کلاس‌های سرویس را در یک فرایند روی پورت‌های loopback برای مجموعه تست بدون داکر بالا می‌آورد.

ساختار گزارش. در بخش‌های ۲ تا ۵ ابتدا شاردینگ و نقشه شارد، سپس انتخاب رپلیکا، کنترل همروندی و در پایان پروتکل 2PC توضیح داده می‌شود. بخش‌های ۶ و ۷ به دو بخش امتیازی مربوط‌اند: مهاجرت زنده به شارد C و کنترلر خودترمیم. بخش ۸ استقرار و ساختار مخزن را نشان می‌دهد، بخش ۹ نتایج آزمون‌ها را گزارش می‌کند و بخش ۱۰ محدودیت‌ها و جمع‌بندی است. کد پروژه در پوشه‌های /platform، /tests و /scripts قرار دارد و ایمیج داکر با نام bank/shard:latest ساخته می‌شود.

ابزارها و فرض‌ها. پیاده‌سازی با Python 3.13 و کتابخانه استاندارد انجام شده است. پایگاه‌داده هر شارد یک فایل SQLite است و Docker Compose فقط برای استقرار و اجرای کانتینرها استفاده می‌شود. برای تست سریع‌تر، یک هارنس محلی هم نوشته شده که همان کلاس‌های سرویس را در یک فرایند اجرا می‌کند. فرض اصلی این است که هر شارد یک پایگاه‌داده مشترک دارد و رپلیکاها موجودی مستقل نگه نمی‌دارند؛ بنابراین رپلیکیشن پایگاه‌داده و failover بین سرورها در این پروژه بررسی نمی‌شود.

مسیر اصلی یک درخواست از کلاینت شروع می‌شود، به گیت‌وی می‌رسد و بعد به یکی از رپلیکاهای شارد مالک می‌رود. در کنار این مسیر، رجیستری و ناظر نقشه برای نگهداری نقشه و اجرای مهاجرت، و کنترلر ترمیم برای جایگزینی رپلیکاها قرار دارند. گیت‌وی و هماهنگ‌کننده را جدا پیاده‌سازی کردم تا منطق 2PC با منطق مسیریابی قاطی نشود؛ هرچند تمرین اجازه می‌داد هر دو در یک کانتینر باشند. همه نقش‌ها از یک ایمیج استفاده می‌کنند تا نسخه کد در همه‌جا یکسان بماند و رفتار رپلیکای جایگزین با رپلیکای اصلی تفاوتی نداشته باشد.

۲. شاردینگ، نقشه شارد و مسیریابی

برای تقسیم داده از پارتیشن‌بندی بازه‌ای استفاده شده است: هر حساب بر اساس شماره‌اش به یک شارد نسبت داده می‌شود و همان شارد مالک آن حساب است. کل داده روی هر شارد کپی نمی‌شود. نقشه شارد در فایل registry.db نگهداری می‌شود، نه در یک فایل پیکربندی. هر بار که نقشه تغییر می‌کند، یک نسخه جدید با اپوک بزرگ‌تر ذخیره می‌شود و اشاره‌گر current_map به آن تغییر می‌کند. آدرس رپلیکاها هم همراه همان اپوک نگهداری می‌شوند. برای اینکه نسخه‌های نقشه در لاگ‌ها قابل تشخیص باشند، یک اثر انگشت کوتاه SHA-256 هم برای هر نقشه ذخیره می‌شود.

اپوک	شارد A	شارد B	شارد C
۱ — اولیه	A0001–A1000	A1001–A2000	وجود ندارد
۲ — پس از بخش امتیازی اول	A0001–A1000	A1001–A1500	A1501–A2000

مسیریابی مراحل ساده‌ای دارد. گیت‌وی ابتدا شکل حساب را بررسی می‌کند، سپس از نقشه جاری shard_of(account) را می‌پرسد، اپوک نقشه را به درخواست اضافه می‌کند و از بین رپلیکاهای همان شارد یک رپلیکا انتخاب می‌کند. تابع ShardMap.owner باریک‌ترین بازه منطبق را برمی‌گرداند تا اگر نقشه هم بازه اصلی و هم زیربازه داشته باشد، زیربازه خاص‌تر انتخاب شود. درخواست یک حساب برای خواندن موجودی هرگز به شارد دیگر فرستاده نمی‌شود؛ مجموع کل پول فقط در تست‌ها و از روی داده هر شارد محاسبه می‌شود.

چون نقشه در پایگاه‌داده نگهداری می‌شود، اضافه‌کردن شارد جدید نیازی به تغییر کد گیت‌وی ندارد. اپراتور رپلیکاهای شارد C را ثبت می‌کند، ناظر نقشه یک اپوک جدید منتشر می‌کند که بازه A1501–A2000 را از B به C منتقل می‌کند و گیت‌وی از درخواست بعدی اپوک جدید را می‌خواند. انتشار نقشه فقط وقتی پذیرفته می‌شود که اپوک جدید از اپوک فعلی بزرگ‌تر باشد. هر تغییر نقشه به همه رپلیکاهای شناخته‌شده فرستاده می‌شود. اگر رپلیکایی در آن لحظه در دسترس نباشد، تا وقتی پاسخ نمی‌دهد نمی‌تواند بنویسد و بعد از برگشتن، نقشه جدید را می‌خواند.

۲.۱ مسیرهای کهنه fence

برای اینکه درخواست‌های قدیمی بعد از تغییر مالکیت مشکلی ایجاد نکنند، خود شارد هم یک نسخه از نقشه را در پایگاه‌داده‌اش (replica_state.shard_map) نگه می‌دارد. قبل از اینکه شارد ردیفی را تغییر دهد، دو چیز را بررسی می‌کند: اگر اپوک درخواست از اپوک محلی شارد قدیمی‌تر باشد، درخواست با stale_epoch 409 رد می‌شود؛ و اگر حساب در آن اپوک متعلق به شارد دیگری باشد، با not_owner 409 رد می‌شود. برای نوشتن، این بررسی داخل همان تراکنش BEGIN IMMEDIATE انجام می‌شود، نه قبل از آن. به این ترتیب درخواستی که هم‌زمان با تغییر مالکیت رسیده باشد یا قبل از fence commit می‌شود یا بعد از آن رد می‌شود. خواندن هم بررسی می‌شود، چون برگرداندن موجودی قدیمی بدتر از رد کردن درخواست است. این بررسی در لایه ذخیره‌سازی (ShardStore.assert_writable) انجام می‌شود، پس اگر کلاینت گیت‌وی را دور بزند و مستقیم به رپلیکا وصل شود هم اثر ندارد.

۳. رپلیکیشن، توزیع بار و دسترس‌پذیری

هر شارد دو رپلیکا دارد که از یک پایگاه‌داده مشترک استفاده می‌کنند: در Compose هر دو کانتینر یک والیوم را mount می‌کنند (shard_a یا shard_b) و در هارنس محلی همان فایل SQLite را باز می‌کنند. رپلیکا موجودی مستقل خودش را نگه نمی‌دارد و فقط یک رابط سرویس‌دهنده روی داده شارد است. به همین دلیل اگر رپلیکاها را هم متوقف کنیم داده از بین نمی‌رود و رپلیکای جایگزین نیازی به کپی داده ندارد. این انتخاب مطابق صورت تمرین است؛ در این پروژه رپلیکیشن پایگاه‌داده و failover بین سرورها پیاده‌سازی نشده و ادعای آن را هم نداریم.

برای انتخاب رپلیکا از استراتژی «کمترین اتصال» استفاده شده است. گیت‌وی برای هر درخواست شمارنده in-flight رپلیکای انتخاب‌شده را یک واحد بالا می‌برد و بعد از پایان درخواست پایین می‌آورد. بین رپلیکاهای سالم، رپلیکایی که کمترین تعداد درخواست در جریان دارد انتخاب می‌شود و برای بارهای مساوی، نوبت به صورت چرخشی عوض می‌شود. این روش برای این پروژه مناسب است چون هزینه عملیات‌ها یکسان نیست؛ مثلاً یک انتقال قفل نوشتن را بیشتر از یک خواندن موجودی نگه می‌دارد. هر پاسخ served_by را برمی‌گرداند تا در تست‌ها مشخص باشد کدام رپلیکا پاسخ داده است.

یک حلقه در پس‌زمینه به صورت دوره‌ای GET /health را روی رپلیکاها صدا می‌زند (هر ۲ ثانیه در Compose و هر ۰٫۵ ثانیه در هارنس محلی). اگر رپلیکایی پاسخ ندهد، از لیست سالم‌ها خارج می‌شود و اگر دوباره پاسخ دهد، برمی‌گردد. اگر یک درخواست به رپلیکای خراب برسد، گیت‌وی آن را روی رپلیکای دیگری از همان شارد دوباره امتحان می‌کند؛ اما خطاهای منطقی مثل موجودی ناکافی یا اپوک قدیمی دوباره تلاش نمی‌شوند. اگر همه رپلیکاها خراب علامت بخورند، گیت‌وی باز هم همه را امتحان می‌کند تا بعد از بازیابی، سیستم سریع برگردد. چون هر نوشتن request_id دارد، تلاش دوباره روی رپلیکای دیگر باعث اعمال دوباره نمی‌شود.

۴. کنترل همروندی، رفتار ACID و پایداری

مبالغ در داخل سیستم به صورت عدد صحیح سِنت نگهداری می‌شوند. عدد اعشاری فقط در ورودی کلاینت پذیرفته و یک‌بار به سِنت تبدیل می‌شود؛ سرویس‌های داخلی با فیلدهای *_cents کار می‌کنند. این کار جلوی خطای گردکردن اعداد اعشاری را می‌گیرد. هر عملیات محلی و هر انتقال درون‌شاردی داخل یک تراکنش BEGIN IMMEDIATE اجرا می‌شود. در این حالت قفل نوشتن قبل از خواندن موجودی گرفته می‌شود و دو رپلیکا نمی‌توانند هم‌زمان یک حساب را بخوانند و تغییر دهند. پایگاه‌داده با journal_mode=WAL، synchronous=FULL و busy_timeout=30000 تنظیم شده است. اگر چند برداشت هم‌زمان برسند، درخواست‌ها به جای خطای SQLITE_BUSY کمی صبر می‌کنند. قید CHECK (balance >= 0) هم به عنوان آخرین محافظ مانع منفی‌شدن موجودی می‌شود و موجودی مبدأ قبل از برداشت در همان تراکنش بررسی می‌شود.

یک قفل درون‌فرایندی (RLock) هم در ShardStore وجود دارد، اما این قفل فقط threadهای یک رپلیکا را مرتب می‌کند. تضمین اصلی بین رپلیکاها قفل نوشتن SQLite است که بین همه فرایندهایی که فایل شارد را باز کرده‌اند مشترک است. انتقال درون‌شاردی هر دو ردیف را در یک تراکنش تغییر می‌دهد؛ یا هر دو موجودی عوض می‌شوند یا هیچ‌کدام. پایداری هم از synchronous=FULL و commit قبل از پاسخ می‌آید؛ یعنی اگر رپلیکا پاسخ «ثبت شد» داده باشد، تغییر روی دیسک نوشته شده است.

هر نوشتن نتیجه‌اش را در جدول applied_requests با کلید request_id ذخیره می‌کند. اگر همان درخواست دوباره برسد، پاسخ ذخیره‌شده با replayed: true برگردانده می‌شود و عملیات دوباره اعمال نمی‌شود. همین جدول یک ستون seq افزایشی هم دارد که داخل تراکنش نوشتن مقدار می‌گیرد؛ این ستون بعداً برای ثبت تغییرات در مهاجرت زنده استفاده می‌شود. یک نکته عملی هم این بود که هر درخواست اتصال SQLite خودش را در پایان آزاد کند؛ وگرنه در بار زیاد تعداد file descriptor هابالا می‌رود و SQLite خطای گمراه‌کننده unable to open database file می‌دهد. برای این موضوع یک تست ساده با ۱۲۰ درخواست نوشته شده است.

۵. تراکنش دوفازی

برای انتقال بین دو شارد، انجام دو به‌روزرسانی جدا کافی نیست؛ چون ممکن است یکی انجام شود و دیگری شکست بخورد. به همین دلیل از پروتکل دوفازی استفاده شده است. مثال صورت تمرین A0500 در شارد A با ۱٬۰۰۰٫۰۰ است که ۳۰۰٫۰۰ به A1500 در شارد B با ۵۰۰٫۰۰ می‌فرستد؛ بعد از commit موجودی‌ها ۷۰۰٫۰۰ و ۸۰۰٫۰۰ می‌شوند. گیت‌وی دو مالک را تشخیص می‌دهد و هماهنگ‌کننده مراحل را اجرا می‌کند.

۵.۱ وضعیت مشارکت‌کننده و پیام‌های تکراری

در سمت شارد، prepare داخل BEGIN IMMEDIATE اجرا می‌شود، مالکیت و حساب را بررسی می‌کند و دو رکورد پایدار می‌نویسد: شاخه در txn_prepared و رأی در prepare_decisions. در این مرحله موجودی تغییر نمی‌کند؛ یعنی prepared مانند یک قول است که اگر بعداً abort شود، هیچ اثری روی داده ندارد. اگر همان PREPARE دوباره برسد، رأی قبلی برگردانده می‌شود و دوباره روی موجودی جدید بررسی نمی‌شود. commit_prepared هم idempotent است: اگر همان تصمیم قبلاً ثبت شده باشد، repeated: true و applied: 0 برمی‌گرداند و اگر تصمیم مخالف ثبت شده باشد، با decision_conflict امتناع می‌کند. تصمیم قبل از پاسخ مشارکت‌کننده ذخیره می‌شود تا اگر بین اعمال و پاسخ کرش رخ داد، بعد از ری‌استارت معلوم باشد چه تصمیمی گرفته شده است.

ژورنال هماهنگ‌کننده برای هر تراکنش یک ردیف دارد و وضعیت‌های INITIATED، PREPARING، PREPARED، COMMIT_DECIDED، ABORT_DECIDED، COMPLETED و ABORTED را نگه می‌دارد. هر تغییر وضعیت یک تراکنش SQLite است، پس بعد از کرش ژورنال در یکی از این وضعیت‌ها قرار دارد و نه بین دو وضعیت. نکته مهم این است که COMMIT_DECIDED قبل از فرستادن اولین پیام COMMIT ذخیره می‌شود. کلاینت فقط وقتی موفقیت می‌بیند که هر دو مشارکت‌کننده COMMIT را تأیید کنند؛ در غیر این صورت پاسخ PENDING می‌گیرد و حلقه بازیابی بعداً کار را تمام می‌کند.

۵.۲ سناریوهای بازیابی

سناریو	وضعیت	پیاده‌سازی و رفتار مشاهده‌شده
A	یک شارد YES می‌دهد، شارد دیگر رد می‌کند (یا در prepare دسترس‌ناپذیر است).	هماهنگ‌کننده ABORT_DECIDED می‌نویسد، فقط شاخه‌هایی که YES داده‌اند را abort می‌کند و ABORTED ثبت می‌کند. هیچ مانده‌ای جابه‌جا نشد. تست: آرا {A: NO, B: YES} ← abort امن، مانده‌ها دست‌نخورده.
B	همه مشارکت‌کنندگان آماده‌اند؛ هماهنگ‌کننده قبل از تصمیم متوقف می‌شود.	پس از ری‌استارت ژورنال PREPARED بدون تصمیم است. هیچ مشارکت‌کننده‌ای چیزی اعمال نکرده، پس بازیابی به همه شاخه‌های آماده ABORT می‌فرستد و ABORTED ثبت می‌کند. تست: شاخه‌های آماده پس از ری‌استارت به ABORT حل شدند.
C	COMMIT ثبت شده اما همه مشارکت‌کنندگان تأیید نکرده‌اند.	پس از ری‌استارت ژورنال COMMIT_DECIDED است؛ بازیابی COMMIT را تا تأیید همه تکرار می‌کند و تنها بعد COMPLETED ثبت می‌کند. تصمیم به‌خاطر down بودن موقت مشارکت‌کننده هرگز به ABORT تبدیل نمی‌شود. تست: COMMIT پایدار پس از ری‌استارت کامل شد و هرگز برنگشت.

تراکنش‌های حل‌نشده در دو جا کوئری‌پذیرند. GET /txn?txn_id=… هماهنگ‌کننده وضعیت ژورنال، آرا، مشارکت‌کنندگان، تعداد تلاش‌ها و پس از یک دوره مهلت PENDING یا PENDING_COMMIT را برمی‌گرداند؛ /2pc/status مشارکت‌کننده PREPARED، COMMITTED، ABORTED یا UNKNOWN را برمی‌گرداند (شاخه آماده قدیمی‌تر از ۳۰ ثانیه به‌جای ادعای قریب‌الوقوع بودن، unknown گزارش می‌شود). کلاینتی که پاسخ را از دست داده با همان txn_id می‌پرسد یا request_id اصلی را replay می‌کند؛ find_by_request هماهنگ‌کننده برای تراکنش کامل نتیجه ذخیره‌شده، برای تراکنش در جریان وضعیت جاری و برای تراکنش abortشده یک خطای صریح برمی‌گرداند.

۶. بخش امتیازی اول — شاردینگ مجدد زنده به شارد C (+۱۲٪)

برای بخش امتیازی اول، شارد C با پایگاه‌داده و دو رپلیکای خودش به سیستم اضافه می‌شود و بعد بازه A1501–A2000 در حین ادامه کار از B به C منتقل می‌شود. شارد C در یک فایل Compose جداگانه تعریف شده تا خوشه اصلی دو شاردی تحت تأثیر قرار نگیرد. مهاجرت به شکل یک ماشین حالت در جدول migrations ذخیره می‌شود. هر فاز قبل از انجام کارش ثبت می‌شود تا اگر اجرا نیمه‌کاره بماند، بشود از همان‌جا ادامه داد.

فاز	کار انجام‌شده	چک‌پوینت پایدار
PLANNED	رزرو اپوک مقصد؛ ثبت بازه، مبدأ، مقصد و اپوک پایه.	ردیف مهاجرت با plan.
SNAPSHOTTING	خروجی ردیف‌های بازه و کرسر درخواست‌های اعمال‌شده در یک خواندن اتمیک از پایگاه‌داده مبدأ.	snapshot_rows، snapshot_position، delta_position.
CATCHUP	نصب ردیف‌ها روی مقصد، سپس replay هر نوشتن بعد از کرسر به‌صورت دسته‌ای؛ سقف دسته ۵۰۰ و پرچم more.	rows_installed، delta_applied، delta_position.
FENCING	حل شاخه‌های آماده تصمیم‌نگرفته روی مبدأ؛ ذخیره نقشه جدید؛ مسلح‌کردن مقصد سپس مبدأ (همه رپلیکاها)؛ drain تا یک pass کاملاً ساکن.	new_map payload، fenced_drain، شناسه تراکنش‌های حل‌شده.
ACTIVATED	انتشار اپوک؛ پوش به همه رپلیکاها؛ گیت‌وی از درخواست بعدی بازه را به C می‌فرستد.	new_epoch، activated_at.
COMPLETED	از مقصد تأیید گرفته می‌شود که بازه را در اپوک مقصد دارد، سپس مبدأ ردیف‌های قدیمی را حذف می‌کند.	completed_at؛ مبدأ تا وقتی مالک بازه است از cleanup امتناع می‌کند.

۶.۱ اسنپ‌شات و ثبت تغییرات مرتب

در مرحله اول، مبدأ با BEGIN IMMEDIATE قفل نوشتن پایگاه‌داده را می‌گیرد و در همان تراکنش ردیف‌های بازه، بزرگ‌ترین seq جدول applied_requests و شاخه‌های آماده را می‌خواند. این کار مهم است چون اسنپ‌شات باید یک پیشوند consistent باشد و کرسر دقیقاً به آخرین نوشتن داخل آن اشاره کند. هر نوشتن seq خودش را در همان تراکنش تغییر موجودی می‌گیرد، پس ترتیب log با ترتیب commit یکی است. در catch-up فقط نوشتن‌هایی replay می‌شوند که حسابشان داخل بازه است و seq آن‌ها از کرسر بزرگ‌تر است. هر ورودی request_id اصلی خودش را همراه دارد و import_applied اول جدول درخواست‌های اعمال‌شده مقصد را نگاه می‌کند؛ بنابراین اگر یک دلتا دوبار replay شود، موجودی فقط یک‌بار تغییر می‌کند. کرسر بعد از هر دسته در رجیستری ذخیره می‌شود.

سایر حساب‌ها در تمام اسنپ‌شات و کچ‌آپ در دسترس می‌مانند. بازه مهاجرت فقط در پوش نهایی fence برای مدت کوتاه unavailable است؛ تمرین دقیقاً همین وقفه کوتاه کات‌اور را مجاز می‌داند. طراحی کل بازه را برای کپی bulk freeze نمی‌کند، از صف در حافظه استفاده نمی‌کند و به کپی یک‌باره تکیه نمی‌کند.

۶.۲ fence، فعال‌سازی و اثرهای exactly-once

ترتیب مراحل fence در این بخش مهم‌ترین قسمت پیاده‌سازی است. اول مبدأ شاخه‌های آماده‌ای را که تصمیم نگرفته‌اند با ABORT امن حل می‌کند؛ شاخه‌ای که تصمیم گرفته شده دست نمی‌خورد. دوم نقشه جدید در progress مهاجرت ذخیره می‌شود تا اگر کرش شد، روی resume همان نقشه استفاده شود. سوم رپلیکاهای مقصد و بعد همه رپلیکاهای مبدأ با نقشه جدید هماهنگ می‌شوند. اگر رپلیکای مقصد در آن لحظه در دسترس نباشد مشکلی ایجاد نمی‌کند، چون تا وقتی پاسخ نمی‌دهد نمی‌تواند بنویسد. بعد از این مرحله، مبدأ بازه را در اپوک‌های قدیمی رد می‌کند. چهارم، تغییرات مبدأ تا جایی خوانده می‌شوند که یک دور کامل چیزی برای اعمال پیدا نکند. در آخر اپوک جدید منتشر می‌شود و ردیف‌های قدیمی فقط بعد از تأیید مالکیت مقصد حذف می‌شوند.

fence عمداً زیر گیت‌وی اعمال می‌شود. تست یک write، یک read و یک transfer را مستقیم به مالک قدیمی با اپوک قبل از مهاجرت می‌فرستد: هر سه 409 stale_epoch می‌گیرند، مانده تغییر نمی‌کند و تلاش کهنه دوم پس از کات‌اور هم شکست می‌خورد. مالک جدید همان حساب را در اپوک جدید می‌پذیرد. این همان اثبات «ارسال عمدی درخواست کهنه پس از کات‌اور» است و حتی با دور زدن گیت‌وی برقرار است.

2PC در جریان قبل و در حین hand-over مدیریت می‌شود. شاخه‌های آماده تصمیم‌نگرفته قبل از تغییر مالکیت روی مبدأ abort می‌شوند؛ هر شاخه‌ای که هنوز آماده بماند با اسنپ‌شات export و روی مقصد با همان import txn_id می‌شود. COMMITی که بعد از کات‌اور برسد توسط مالک جدید و فقط آن اجرا می‌شود. تصمیم COMMIT پایدار هرگز دور ریخته نمی‌شود. تست یک بدهکار روی B برای A1700 آماده می‌کند، بازه را مهاجرت می‌دهد و سپس از C commit می‌کند: مالک جدید دقیقاً یک‌بار اعمال می‌کند و مالک قدیمی دیگر شاخه را ندارد. انتقال‌های بین‌شاردی در طول کات‌اور با حفظ پول ادامه می‌یابند.

۶.۳ سازگاری با کرش

بازیابی با فاز پایدار هدایت می‌شود، نه با replay یک اسکریپت. کرش در **snapshot** اسنپ‌شات را دوباره شروع می‌کند (پیشوند تازه همیشه امن است)؛ کرش در **catch-up** اگر دسته در حافظه از دست رفته باشد ردیف‌ها را دوباره export می‌کند و از delta_position پایدار ادامه می‌دهد؛ کرش در **fencing** از new_map ذخیره‌شده استفاده می‌کند و همه رپلیکاها را دوباره مسلح و drain می‌کند؛ کرش پس از **activation** و قبل از cleanup می‌بیند اپوک قبلاً منتشر شده، تأیید می‌کند مقصد هنوز مالک بازه است و تنها بعد ردیف‌های قدیمی را حذف می‌کند. ناظر یک حلقه resume پس‌زمینه هر ۳ ثانیه و همچنین در startup اجرا می‌کند. تست یک اجرای کامل از SNAPSHOTTING و سپس resume از CATCHUP، FENCING و ACTIVATED را انجام می‌دهد؛ هر موتور تازه با حفظ پول همگرا می‌شود. نقشه نهایی B را با ۵۰۰ حساب و C را با ۵۰۰ حساب دارد، A1500 روی B و A1600/A1999 روی C می‌مانند.

دو مشکلی که در آزمون‌ها دیدم. زیر بار زنده، ابتدا یکی دو واریز گم می‌شد. علت این بود که خواندن ردیف‌های اسنپ‌شات و خواندن کرسر دو خواندن جدا بودند و یک رپلیکای دیگر می‌توانست بین آن‌ها commit کند. بعد از اینکه هر دو خواندن را داخل یک قفل نوشتن پایگاه‌داده قرار دادم، مشکل حل شد. مشکل دوم این بود که fence فقط به یک رپلیکا فرستاده می‌شد و رپلیکای دیگر هنوز نوشتن بازه در حال خروج را می‌پذیرفت. بعد از فرستادن fence به همه رپلیکاها و drain تا یک دور کاملاً ساکن، این مشکل هم برطرف شد. بعد از این تغییرات، تست ترافیک زنده ۱۰۵ عملیات را انجام داد و ۵٬۲۵۰ سِنت تأییدشده حفظ شد.

۷. بخش امتیازی دوم — خودترمیمی خودمختار (+۱۰٪)

platform/selfhealing.py یک برنامه پایتون ساده است که به عنوان کنترلر خودترمیم کار می‌کند. این کنترلر از supervisor، Docker Swarm، Kubernetes آماده یا سیاست restart داکر استفاده نمی‌کند و خودش تصمیم می‌گیرد کدام رپلیکا خراب است. در هر دور، لیست رپلیکاهای ثبت‌شده را از رجیستری می‌خواند، health/ هر کدام را صدا می‌زند و برای هر شاردی که تعداد رپلیکاهای سالمش کمتر از تعداد مطلوب باشد، یک جایگزین روی همان پایگاه‌داده شارد بالا می‌آورد و آدرس آن را در نقشه ثبت می‌کند.

دو نکته باعث می‌شود این کار امن باشد. اول اینکه جایگزین روی همان پایگاه‌داده شارد اجرا می‌شود، نه روی یک کپی جدا؛ بنابراین دو نسخه متناقض از موجودی ایجاد نمی‌شود و نیازی به انتقال داده هم نیست. دوم اینکه قبل از ساخت جایگزین، یک claim در رجیستری ثبت می‌شود. اگر کنترلر در میانه کار کرش کند، کنترلر بعدی claim ناتمام را می‌بیند و برای همان شکاف دو رپلیکا نمی‌سازد. اگر ساخت موفق شود claim کامل می‌شود و اگر ساخت شکست بخورد، claim آزاد می‌شود تا دور بعد دوباره تلاش کند. این مکانیزم از زیادشدن تعداد رپلیکاها بعد از چند خرابی جلوگیری می‌کند.

در Compose، کنترلر برای ساخت کانتینر جدید از Docker API استفاده می‌کند و همان والیوم شارد را به کانتینر می‌دهد؛ بنابراین رپلیکای جدید به همان پایگاه‌داده قبلی وصل می‌شود. نام کانتینرهای جایگزین جدا انتخاب می‌شود تا با کانتینرهای Compose تداخل نداشته باشند. کانتینرهای در حال اجرا حذف نمی‌شوند و فقط باقی‌مانده‌های متوقف‌شده پاک می‌شوند. اگر Docker درخواست را رد کند، پیام خود Docker به کنترلر برمی‌گردد و در لاگ ثبت می‌شود.

کنترلر با شش تست بررسی می‌شود: نگه‌داشتن دو رپلیکا در حالت سالم، جایگزینی رپلیکای کشته‌شده، جلوگیری از ساخت تکراری بعد از کرش کنترلر، درستی تراکنش‌ها زیر بار هم‌روند، نمایش خطای Docker و mount شدن والیوم واقعی شارد. در تست بار هم‌روند، در مجموع ۸۱۹ عملیات انجام شد و در ۶ بار جایگزینی، ۸۱٬۹۰۰ سِنت تأییدشده حفظ شد. تست‌های مربوط به Docker socket روی ویندوز skip می‌شوند و روی ماشین اوبونتو اجرا می‌شوند.

۸. استقرار، بهره‌برداری و مخزن

استقرار پروژه با Docker Compose انجام می‌شود. یک ایمیج بر پایه python:3.13-slim ساخته می‌شود و همه نقش‌ها با متغیر ROLE انتخاب می‌شوند. هر شارد یک والیوم جدا دارد، رجیستری و ژورنال هماهنگ‌کننده هم والیوم خودشان را دارند. فقط گیت‌وی، هماهنگ‌کننده و ناظر نقشه پورت منتشر می‌کنند. فایل Compose عمداً هیچ restart: ندارد، چون بازیابی کار کنترلر است.

سرویس(ها)	پورت	وضعیت پایدار / نقش
shard-a-1 / shard-a-2	۸۰۸۰ داخلی	والیوم shard_a، data/shard-A.db/؛ دو فرانت‌اند روی پایگاه‌داده A.
shard-b-1 / shard-b-2	۸۰۸۰ داخلی	والیوم shard_b، data/shard-B.db/؛ دو فرانت‌اند روی پایگاه‌داده B.
shardmap	۸۵۰۰	والیوم registry، registry.db؛ نقشه، اپوک‌ها، اکتورها، وضعیت مهاجرت و claimها.
gateway	۸۰۰۰	رجیستری را می‌خواند؛ مسیریابی مالک و انتخاب رپلیکا با کمترین اتصال.
coordinator	۸۱۰۰	والیوم coordinator، coordinator.db؛ ژورنال 2PC و حلقه بازیابی.
selfhealer	—	والیوم رجیستری به‌همراه var/run/docker.sock/؛ بررسی، spawn، claim و ثبت.
shard-c-1 / shard-c-2 (overlay)	۸۰۸۰ داخلی	والیوم shard_c؛ فقط برای بخش اول اضافه می‌شود و هرگز بخشی از خوشه پایه نیست.

نقاط ورود اپراتور bash scripts/up.sh (ساخت، شروع، انتظار سلامت، ثبت رپلیکاها و پوش نقشه)، bash scripts/demo_migration.sh (افزودن شارد C و انتقال بازه زیر بار پس‌زمینه)، bash scripts/demo_selfhealing.sh (کشتن یک رپلیکا و مشاهده heal) و python -m tests.run_tests (کل مجموعه بدون داکر) هستند. هارنس محلی همان کلاس‌های سرویس را روی پورت‌های loopback بالا می‌آورد، پس رفتار پروتکل سریع تست می‌شود؛ جایگزین استقرار Compose نیست، که همان نسخه امتیازی است. docs/RUNNING_ON_UBUNTU.md راهنمای گام‌به‌گام VM و docs/VIDEO_SCRIPT.md اسکریپت دموست.

مسیر	محتوا
platform/	client.py ،selfhealing.py ،migration.py ،shardmap_service.py ،coordinator.py ،gateway.py ،shard_server.py .targets.py ،http_service.py ،http_api.py ،local_stack.py
platform/storage/	shard_store.py (حساب‌ها، idempotency، شاخه‌های آماده، fence) و registry.py (نقشه شارد، وضعیت مهاجرت، claimها، ژورنال هماهنگ‌کننده).
platform/sharding/	shard_map.py: بازه‌ها، اپوک، جست‌وجوی باریک‌ترین مالک، اثر انگشت، vacate_subrange.
tests/	۲۷ بررسی در چهار گروه experiments، recovery، bonus1 و bonus2، به‌همراه helpers.py برای تولید ترافیک و اجرای تست.
docker/ ،scripts/	بالا آوردن استک، هر دو دموی امتیازی، هارنس محلی/تست و run.sh توزیع نقش برای ایمیج واحد.
docker-compose.yml، docker-compose.migrate.yml، Dockerfile	خوشه پایه، overlay شارد C و ایمیج تک‌کتابخانه‌ای استاندارد.

۹. نتایج

مجموعه تست خودکفا است و از همان کلاس‌های سرویس Compose استفاده می‌کند. اجرای نهایی **۲۷/۲۷ بررسی را در ۱۲۲٫۵ ثانیه قبول کرد.** روی Windows 11 با Python 3.13 اجرا شد؛ سه بررسی فقط-Linux (/proc/self/fd) و دو تست Docker socket با AF_UNIX) آنجا skip می‌شوند و روی VM اوبونتو اجرا می‌شوند. نتایج زیر پیام‌های واقعی همان اجراست.

#	آزمایش الزامی	آنچه مشاهده شد
۱	توزیع شارد	شارد A دقیقاً ۱۰۰۰ حساب (A0001–A1000) و B دقیقاً ۱۰۰۰ (A1001–A2000) داشت، بدون هم‌پوشانی. مسیر موجودی: A0500 → B، A1500 → A، A0999؛ debug route بازه و دلیل را نام می‌برد.
۲	توزیع بار رپلیکا	۲۴ درخواست موجودی به A0500 بین دو رپلیکای A تقسیم شد: **۱۲/۱۲** هر پاسخ served_by داشت و شمارنده‌های گیت‌وی هم‌خوان بودند.
۳	عملیات هم‌روند روی رپلیکاها	۵۰ واریز هم‌روند ۱۳۷ سِنت از طریق هر دو رپلیکا دقیقاً **۱۶۶٬۷۵۰ سِنت** (مورد انتظار ۱۶۶٬۷۵۰) تولید کرد؛ بدون lost update.
۴	قطع رپلیکا	یک رپلیکای A زیر بار کشته شد. گیت‌وی آن را unhealthy علامت زد؛ **۹۸ عملیات** روی رپلیکای بازمانده بدون خطای گذرا ادامه یافت و مانده نهایی دقیق بود.
۵	انتقال درون‌شاردی و بین‌شاردی	انتقال درون‌شاردی از A0500 به A0600 با ۲۵٫۰۰ اتمیک، coordinated:false و بدون txn_id بود. انتقال بین‌شاردی از A0500 به A1500 با ۳۰٫۰۰ با coordinated:true، آرا {A:YES,B:YES}، ژورنال COMPLETED و مشارکت‌کنندگان COMMITTED بود.
۶	تراکنش توزیع‌شده قطع‌شده	هر دو شاخه آماده شدند، فقط A commit شد و ژورنال COMMIT_DECIDED بود؛ پس از ری‌استارت بازیابی B را کامل کرد و ژورنال COMPLETED با تصمیم همچنان COMMIT شد. بازیابی دوم چیزی جابه‌جا نکرد؛ تراکنش تصمیم‌نگرفته به‌صورت PREPARED کوئری‌پذیر بود.
۷	انتقال‌های هم‌روند بین‌شاردی	۲۴ انتقال هم‌پوشان بین A↔B با top-upهای شمارش‌شده، پول را در **۲۹۹٬۹۱۲٬۰۰۰ سِنت** حفظ کرد؛ هر success با coordinated:true و روی هر دو مشارکت‌کننده commit شده بود؛ replay همان request_id همان تراکنش را برگرداند و پول را جابه‌جا نکرد.

گروه	تعداد	نتیجه
experiments	۸	۷ آزمایش الزامی + گارد رگرسیون آزادسازی descriptor؛ همه pass (گارد خارج Linux skip می‌شود).
recovery	۸	سناریوهای A/B/C، COMMIT تکراری، PREPARE تکراری، کوئری حل‌نشده، پایداری پس از restart کامل و startup شارد؛ همه pass.
bonus1	۵	کپی آنلاین + دلتا؛ fence مسیر کهنه (اپوک ۱←۲)؛ 2PC در کات‌اور؛ resume از چهار فاز؛ ترافیک زنده (۱۰۵ عملیات، +۵٬۲۵۰ سِنت حفظ‌شده).
bonus2	۶	نگهداشتن ۲ رپلیکا؛ جایگزینی رپلیکای مرده (مانده ۱۷۵٬۲۰۰ سِنت سالم)؛ بدون تکراری پس از کرش کنترلر؛ ۸۱۹ عملیات در ۶ heal با حفظ ۸۱٬۹۰۰ سِنت؛ بررسی رد Docker و mount والیوم (Linux).

۱۰. محدودیت‌ها و نتیجه‌گیری

این پروژه در حد یک تمرین درس طراحی شده و طبیعتاً محدودیت‌هایی دارد. رپلیکیشن در اینجا رپلیکیشن برنامه روی یک پایگاه‌داده مشترک در هر شارد است، نه رپلیکیشن پایگاه‌داده؛ بنابراین والیوم و هاست هر شارد یک نقطه شکست واحد هستند و failover بین ماشین‌ها وجود ندارد. SQLite نوشتن‌ها را در هر شارد سریال می‌کند؛ این کار برای درستی تراکنش‌ها خوب است اما سرعت نوشتن را محدود می‌کند. ژورنال هماهنگ‌کننده هم یک فایل SQLite است و خودش رپلیکا نمی‌شود. تعداد حساب‌ها و بازه‌ها ثابت است، موتور مهاجرت در هر لحظه یک مهاجرت را اداره می‌کند و کنترلر ترمیم روی همان هاست اجرا می‌شود. احراز هویت، TLS و جداسازی چندمستاجری هم در این پروژه وجود ندارد.

با وجود این محدودیت‌ها، بخش‌های اصلی تمرین پیاده‌سازی و آزمایش شده‌اند: شاردینگ بازه‌ای با نقشه اپوکی، دو رپلیکا در هر شارد، مسیریابی با کمترین اتصال، کنترل همروندی با تراکنش‌های SQLite، پروتکل 2PC با تصمیم پایدار و بازیابی، مهاجرت زنده به شارد C و کنترلر خودترمیم. مجموعه ۲۷ تست پروژه در اجرای نهایی همه قبول شدند. مهم‌ترین درسی که از این پروژه گرفتم این بود که جزئیات کوچک، مثل ترتیب خواندن اسنپ‌شات و کرسر یا فرستادن fence به همه رپلیکاها، می‌توانند تفاوت بین یک پیاده‌سازی درست و یک پیاده‌سازی ظاهراً درست باشند.

پیوست الف. رابط HTTP انتخاب‌شده

سطح	Endpoint	هدف
کلاینت ← گیت‌وی	GET /account, POST /deposit, POST /withdraw, POST /transfer	تنها API بانکی لازم؛ همه مسیریابی داخلی است.
کلاینت ← گیت‌وی	GET /shard-map, GET /txn?txn_id=, GET /gateway/stats	بررسی مالکیت/اپوک، کوئری نتیجه نامعلوم و دیدن بار رپلیکا.
هماهنگ‌کننده	POST /txn, GET /txn?txn_id=, GET /txn/unresolved, POST /txn/recover	شروع انتقال بین‌شاردی، کوئری وضعیت، بررسی یا اجرای بازیابی.
رپلیکای شارد	POST /read, /write, /transfer, /2pc/prepare|commit|abort|status	مسیر داده مشارکت‌کننده و پروتکل 2PC.
رپلیکای شارد (داخلی)	POST /internal/shard-map, /internal/migrate/export|import|delta|apply|verify|cleanup	مسلح‌کردن fence، گرفتن اسنپ‌شات، replay دلتا، تأیید و پاک‌سازی.
ناظر	POST /migration/start|resume|cleanup, GET /migrations, POST /shards/register|set, GET /shard-map/history	اداره نقشه، افزودن شارد، اجرا یا ادامه مهاجرت و مشاهده تاریخ.

پیوست ب. بازتولید ارزیابی

۱. bash scripts/up.sh — ساخت و شروع استک پایه، انتظار سلامت، ثبت هر دو رپلیکای A و B و پوش نقشه. خط آخر باید replicas registered: A:2,B:2 را نشان دهد.

۲. python -m tests.run_tests — اجرای هر ۲۷ بررسی بدون داکر؛ افزودن experiments، recovery، bonus1 یا bonus2 برای اجرای یک گروه.

۳. bash scripts/demo_migration.sh — افزودن شارد C، واریز پس‌زمینه به A1800، انتقال A1501-A2000 از B به C، پاک‌سازی و چاپ مسیر جدید و تاریخ نقشه.

۴. bash scripts/demo_selfhealing.sh — بازگرداندن B به جفت Compose، کشتن یک رپلیکا، مشاهده بازگشت دو رپلیکای زنده و واریز از شارد درمان‌شده.

۵. بررسی دستی: python -m platform.client --gateway http://localhost:8000 balance A0500؛ هر پاسخ شارد و رپلیکای سرویس‌دهنده را نشان می‌دهد.